PART 3

Claude Fundamentals

Six chapters that turn the model from an object of study into a component you name, configure and hold to a budget: the family and its versioning, how to choose, the Messages API, the input modalities, thinking as a parameter, and the limits you will meet in production.

IN THIS PART

- 14. The Claude Model Family
- 15. Choosing a Model
- 16. The Messages API
- 17. Streaming, Vision, Files and Documents
- 18. Extended and Adaptive Thinking
- 19. Rate Limits, Pricing and Quotas

CHAPTER 14

The Claude Model Family

In this chapter

Introduction: Seven Days . . . 4
14.1 The Family, as at July 2026 . . . 5
14.2 The Tiers Are a Shape, Not a Ladder . . . 6
14.3 Model IDs and Versioning . . . 7
14.4 Frozen Weights, Moving Infrastructure . . . 8
14.5 Limits: Context, Output and the Batch Exception . . . 9
14.6 Two Cutoffs, and Why the Distinction Exists . . . 9
14.7 Money . . . 9
14.8 The Feature Matrix . . . 10
14.9 One Model, Five Front Doors . . . 11
14.10 The Lifecycle . . . 12
14.11 Deprecation Has Costs, and Anthropic Says So . . . 12
14.12 Asking the Platform Instead of a Table . . . 13
14.13 Visual Analogies . . . 13
14.14 Architecture: The String Is the Bug . . . 14
14.15 Examples: Four Strings, Four Verdicts . . . 15
14.16 Code Walkthrough: A Registry That Refuses to Guess . . . 16
14.17 Mini Project: The Model Registry and Preflight . . . 17
14.18 Panel C — preflight over requests a real team would send . . . 19
14.19 Panel D — the one-line migration, expanded . . . 20
14.20 Panel E — one workload, four models . . . 21
14.21 Evaluation and deployment . . . 21
14.22 Hands-on Labs . . . 22
14.23 Lab Solutions . . . 22
14.24 Best Practices . . . 23
14.25 Common Mistakes . . . 23
14.26 Performance Considerations . . . 25
14.27 Security Considerations . . . 25
14.28 Real Production Examples . . . 26

14.29 Certification Notes 26
14.30 Twenty Practice Questions 27
14.31 Answers and Explanations 28
14.32 Ten Practical Exercises 29
14.33 Five Challenge Labs 30
14.34 Ten Interview Questions 30
14.35 Summary 31
14.36 Glossary 32
14.37 Further Reading 33

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Name the current Claude models, their positions in the family, and what each one is described as being for.
2. Read a **model ID** correctly — dated versus dateless, snapshot versus alias — and explain why "dateless means latest" is the most expensive misconception in the platform.
3. Distinguish **frozen weights** from **serving infrastructure**, and say which one explains behavioural drift on a model ID you never changed.
4. State the **context, output, pricing and knowledge-cutoff** figures for the current family, and where the batch output beta changes them.
5. Explain why **capability does not travel in a bundle**, and read a feature matrix rather than assuming a tier ordering.
6. Describe how the same model is addressed on the **Claude API, Amazon Bedrock, Claude Platform on AWS, Google Cloud and Microsoft Foundry**, and whose retirement calendar governs each.
7. Use the **model lifecycle** vocabulary — active, legacy, deprecated, retired — and run a deprecation audit before a retirement date runs you.
8. Build a **model registry with provenance**, so that every fact your system depends on is either verified or explicitly unknown.

NOTE

Prerequisites

Chapter 9 for what a model actually is — weights, checkpoint, deployment, alias — because this chapter is that vocabulary applied to a real catalogue. **Chapter 11** for the rate card and the cache multipliers; every price here is a line in that model. **Chapter 12** for why parameter count is not the tier ordering it looks like. **Chapter 13** for thinking modes, `effort`, task budgets and the compatibility rules, all of which vary by model and are half of what this chapter tabulates.

Part 3 begins here. Chapters 1 to 13 were about what a language model is, what it cannot do, what it costs and how it spends compute. From this point the model is a component you name in a request, and naming it correctly is a surprisingly large part of the job. Chapter 15 turns this chapter's catalogue into a selection procedure; this chapter is the catalogue, the versioning rules and the clock.

Introduction: Seven Days

It is Wednesday 29 July 2026. Somewhere, a team has seven days.

Their document-processing service has run on `claude-opus-4-1-20250805` since it launched. The model was deprecated on 5 June, and the retirement date is 5 August. Anthropic sent the email — the documented commitment is at least sixty days' notice before retirement for publicly released models, and that commitment was met. The email went to the account owner, who had left in March. The distribution list was never updated. Nobody on the team that owns the service has read the deprecation page since they wrote the integration.

On 5 August their requests will fail. Not degrade: fail.

That is the first failure and it is boring, in the way that most production failures are boring. The second one is more interesting, because it is the failure that happens *while fixing the first*.

An engineer finds the deprecation notice, sees the recommended replacement, and reasons — sensibly — that if they are changing the model string anyway they may as well move to something current. They change one line from `claude-opus-4-1` to `claude-sonnet-5`, run the test suite, and it passes, because the test suite mocks the API. In staging, four things go wrong at once. The `temperature: 0.3` that has been in the request builder since 2024 now returns a 400 on every request. The `thinking: {"type": "enabled", "budget_tokens": 8000}` block returns a 400 as well. The JSON prefill they use to force well-formed output fails, because thinking is on by default on the new model and you cannot prefill while thinking is on. And the monthly cost projection, which nobody re-ran, is wrong in a way that will be corrected upward on 1 September when Sonnet 5's introductory pricing ends.

None of those four are bugs in anybody's code. They are all documented, all discoverable, and all invisible in a diff that changed eleven characters.

This chapter is about the gap between those two facts. A model is named by a string, and a string is the least informative object in your codebase: it carries no limits, no prices, no capabilities, no expiry date, and no warning when any of those change. The whole of the chapter's practical content is a single move — **turn the string into a record** — and the mini project builds the record, with a validator that would have caught all four failures above before the request left the building. It catches all four, in sections 14.18 to 14.20, and prints the reasons.

14.1 The Family, as at July 2026

Four models are generally available on the Claude API. Everything in this section was verified against `platform.claude.com/docs/en/about-claude/models/overview` on 29 July 2026, and all of it changes; treat the shape as durable and the numbers as perishable.

	Claude Fable 5	Claude Opus 5	Claude Sonnet 5	Claude Haiku 4.5
Described as	Next-generation intelligence for long-running agents	For complex agentic coding and enterprise work	The best combination of speed and intelligence	The fastest model with near-frontier intelligence
API ID	`claude-fable-5`	`claude-opus-5`	`claude-sonnet-5`	`claude-haiku-4-5-20251001`
Context	1M	1M	1M	200K
Max output	128K	128K	128K	64K
Price in/out per MTok	$10 / $50	$5 / $25	$2 / $10 †	$1 / $5
Reliable cutoff	Jan 2026	May 2026	Jan 2026	Feb 2025

	Claude Fable 5	Claude Opus 5	Claude Sonnet 5	Claude Haiku 4.5
Training cutoff	Jan 2026	May 2026	Jan 2026	Jul 2025
Latency	Slower	Moderate	Fast	Fastest

† Sonnet 5's listed price is $3 / $15; introductory pricing of $2 / $10 applies through 31 August 2026.

The documentation's own guidance, quoted rather than paraphrased because it is the sort of thing an exam asks: if you are unsure which model to use, start with **Claude Opus 5** for complex agentic coding and enterprise work; for workloads that need the highest available capability, use **Claude Fable 5**. All current models support text and image input, text output, multilingual capability and vision.

Two things about that table are worth stopping on before the rest of the chapter.

Haiku 4.5 is a generation behind, and that is not an oversight. It is the only current model with a 200K context rather than 1M, the only one at 64K output rather than 128K, and the only one whose reliable knowledge cutoff — February 2025 — is over a year before the others. It is also the only one that costs a dollar per million input tokens. Chapter 12 explained why a small model can be far more capable than its size suggests: it is trained far past compute-optimal precisely so that inference is cheap. Haiku 4.5 is that argument shipped.

The price ladder is exact. $1, $2, $5, $10 in; $5, $10, $25, $50 out. Output is five times input on every model in the family, without exception, which is the fact Chapter 13's economics rested on and which the mini project asserts as a test.

14.1.1 Project Glasswing

Two further models exist and are not generally available. **Claude Mythos 5** (`claude-mythos-5`) shares Fable 5's specifications and pricing, and joins the invitation-only **Claude Mythos Preview** (`claude-mythos-preview`) within Project Glasswing. The documentation states that these are offered separately for **defensive cybersecurity workflows**, that access is invitation-only, and that there is no self-serve sign-up; access goes through an Anthropic, AWS or Google Cloud account team. Mythos Preview is itself now deprecated in favour of Mythos 5.

They appear in this chapter for three reasons. They show up in the capability tables you will read, so you need to know why you cannot use them. They are a reminder that "the model family" is not identical to "the models you can call". And they are the clearest available illustration that release is a policy decision rather than a purely technical one — a theme Chapter 61 picks up under governance.

14.2 The Tiers Are a Shape, Not a Ladder

It is tempting to read Fable → Opus → Sonnet → Haiku as a single quality axis with a price attached. That reading survives contact with the table above and does not survive contact with the feature matrix.

DEFINITION

Model tier

A **tier** is a name for a family position — Fable, Opus, Sonnet, Haiku — that indicates roughly where a model sits on the capability/latency/price trade-off. It is a marketing and design axis, not a guarantee about any specific capability, and it does not imply that a higher tier supports every feature a lower one does.

Here is the counterexample, and it is not a corner case. **Claude Sonnet 5 supports adaptive thinking and all five effort levels, and does not support task budgets. Claude Opus 5 supports all three.** Both are current, both are 5-series, both have 1M context and 128K output. A design that assumed "Sonnet 5 is Opus 5 with less of everything" would have shipped a task-budget integration that 400s on half its traffic.

Two more, from Chapter 13's material:

- **Claude Haiku 4.5 is the only current model that uses extended thinking**, the mode the documentation now labels legacy. It also does not support the `effort` parameter at all, and does not support interleaved thinking. A "just swap in Haiku for the cheap path" refactor is four code changes, not one.
- **Claude Opus 4.5** — still active, retirement floor November 2026 — supports `effort` *and* is extended-thinking-only, which is a combination none of the current four has. The documentation gives it a special rule: set both `effort` and `budget_tokens`.

KEY TAKEAWAYS

Capability does not travel in a bundle

Read the matrix, not the tier. Every feature — thinking mode, effort levels, task budgets, interleaved thinking, batch output beta, sampling-parameter acceptance, thinking-block preservation — has its own per-model support list, and those lists do not nest. This is the single most reliable source of avoidable 400s in the platform, and the reason the mini project is a table rather than a paragraph.

14.3 Model IDs and Versioning

This section contains the most consequential piece of knowledge in the chapter.

Every Claude model ID identifies a **pinned version**. When you use a model ID in a request, the underlying model remains constant for the lifetime of that ID. Anthropic does not update the weights or configuration of an existing model ID; an updated version ships under a new ID.

There are two ID formats, and the split is the 4.6 generation.

From the 4.6 generation onward — dateless:

```
claude-{name}-{major}[-{minor}]
claude-sonnet-4-6   claude-opus-4-7   claude-opus-4-8
claude-sonnet-5     claude-opus-5     claude-fable-5
```

Major releases such as Sonnet 5 and Opus 5 omit the minor segment.

Before the 4.6 generation — dated:

```
claude-{name}-{major}-{minor}-{YYYYMMDD}
claude-sonnet-4-5-20250929   claude-haiku-4-5-20251001
```

And now the misconception, stated by the documentation as a misconception because it is common enough to warrant a heading of its own:

> WARNING
>
> **A dateless ID is a snapshot, not a pointer**
>
> `claude-sonnet-4-6` and `claude-sonnet-5` are **not** evergreen aliases that route to the latest or best-performing version. For the 4.6 generation and later, the dateless ID *is* the canonical model ID for that release, mapping to a single fixed snapshot.
>
> The confusion comes from the older generation, where the Claude API *does* offer convenience aliases. `claude-sonnet-4-5` is a pointer that resolves to the most recent dated snapshot for that minor version. `claude-sonnet-5` is not a pointer. It has no dated form to point at.
>
> The practical consequence is asymmetric and unforgiving. If you believe dateless means "latest", you will assume `claude-opus-5` will silently improve — it will not, and you will miss the next model entirely. If you believe dateless means "pinned" and apply that to `claude-sonnet-4-5`, you will believe you have pinned something that can move under you.
>
> Every model ID, dated or dateless, has its **own distinct deprecation and retirement schedule**.

The mini project encodes exactly this distinction: `resolve()` treats pre-4.6 short forms as pointers and 4.6-generation dateless IDs as snapshots, and there is a test asserting that `claude-sonnet-5` does not appear in the alias map. It is a two-line test and it protects a class of production surprise.

14.4 Frozen Weights, Moving Infrastructure

Chapter 9 separated four things people conflate: model, checkpoint, deployment and alias. This section is the operational consequence, and the documentation states it plainly enough to quote.

Model weights are fixed for a given ID. The **serving infrastructure around the model can change** — the request router, safety classifiers, sampling logic. Occasionally an infrastructure update produces minor differences in observable behaviour even when the model ID and weights have not changed. If you notice unexpected behavioural differences on a previously stable model ID, an infrastructure update is the **most likely cause**.

Three things follow, and they are the shape of every "the model changed on us" incident.

Pinning fixes weights, not behaviour. This is a real guarantee and a bounded one. It rules out one class of change and not the others.

Your regression suite is the only instrument that will notice. Chapter 8 built evaluation sets and Chapter 24 turns them into regression suites; this is what they are for. A behavioural drift that is invisible in your logs and visible in a fifty-example evaluation set is the normal case, not the exotic one.

Escalate with evidence, not adjectives. "It feels worse" is unactionable. "Our fifty-example suite went from 46/50 to 39/50 on this date, on this pinned ID, with these three failing examples" is a support ticket somebody can work.

14.5 Limits: Context, Output and the Batch Exception

Two numbers per model, and one exception that catches people.

Context window is the total input the model will accept: 1M tokens on Fable 5, Opus 5 and Sonnet 5; 200K on Haiku 4.5. Chapter 6 explained why a token is not a word and Chapter 9 explained what a million-token KV cache costs in memory; Chapter 52 is where the context *budget* becomes a design discipline. For now the operational point is that a 1M window is the default with no beta header and at standard pricing.

Max output is the ceiling on generated tokens in one response: 128K on the three 5-series models, 64K on Haiku 4.5. Chapter 13's rule stands — thinking tokens count toward `max_tokens` along with visible text, so a 128K ceiling is shared between reasoning and answer.

The exception: on the **Message Batches API**, the `output-300k-2026-03-24` beta header raises the limit to **300K output tokens** on Claude Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6. Note who is missing from that list. Not Fable 5, and not Haiku 4.5. This is section 14.2's rule again: the highest-capability model in the family does not support the largest output ceiling in the family.

14.6 Two Cutoffs, and Why the Distinction Exists

The documentation lists two dates per model and they mean different things.

> DEFINITION
>
> **Reliable knowledge cutoff and training data cutoff**
>
> The **reliable knowledge cutoff** is the date through which a model's knowledge is most extensive and reliable. The **training data cutoff** is the broader date range of training data used.

For Fable 5, Opus 5 and Sonnet 5 the two dates coincide. For **Claude Haiku 4.5 they do not**: reliable knowledge cutoff February 2025, training data cutoff July 2025. The gap is five months, and the honest reading is that data from the tail of a training run is present but thinner, so knowledge from that window is less dependable than knowledge from the bulk of the corpus.

Why does this matter architecturally? Because it is the cleanest possible statement of a limit Chapter 10 called structural. A model's knowledge of the world ends on a date. It does not degrade politely past that date and it does not know that it has stopped. Every question about current state — prices, incumbents, versions, whether a service still exists — is a retrieval problem, not a model-capability problem, and Chapter 53 is where retrieval gets built properly. The cutoff table is where you find out how large that problem is for the model you have chosen.

14.7 Money

Chapter 11 built the cost model; this section supplies its coefficients and one date.

Model	Input $/MTok	Output $/MTok	Batch	1M requests, 4K in / 600 out
Claude Fable 5	10	50	−50%	$70,000
Claude Opus 5	5	25	−50%	$35,000
Claude Sonnet 5	2 †	10 †	−50%	$14,000
Claude Haiku 4.5	1	5	−50%	$7,000

The right-hand column is one representative workload — a million requests a month, four thousand input tokens and six hundred output tokens each, no caching — priced by the mini project's `cost()` function in section 14.20. The spread from Haiku to Fable is exactly ten-fold, and no part of that ratio is an estimate.

Three additions to Chapter 11's arithmetic, all still current:

- **Prompt caching**: 1.25× for a five-minute write, 2× for a one-hour write, **0.1× for a read**. A cached prefix is the single largest lever on the input side.
- **Batch API**: 50% off input *and* output.
- `inference_geo: "us"`: a 1.1× multiplier on every category.

And the date. **Sonnet 5's $2 / $10 is introductory pricing, applying through 31 August 2026; the standard rate is $3 / $15.** For the workload above that is $14,000 a month becoming **$21,000** on 1 September — a 50% increase already on the calendar, requiring no change from anyone. A cost model built in July that does not carry that date will be wrong in September, and it will look like a leak rather than a scheduled event. The mini project prints both figures side by side because a price with an expiry is a different object from a price.

14.8 The Feature Matrix

Everything Chapter 13 established about thinking, restated per model, because this is where the 400s come from. Verified 29 July 2026.

Feature	Fable 5	Opus 5	Sonnet 5	Haiku 4.5
Adaptive thinking	Yes, always on	Yes	Yes	No
Extended thinking (`type: "enabled"`)	No — 400	No — 400	No — 400	**Yes, only mode**
Thinking on by default	Yes	Yes	Yes	No
Can disable thinking	**No**	At effort ≤ `high`	Yes	Yes
`display` default	`omitted`	`omitted`	`omitted`	`summarized`
`effort` parameter	All five	All five	All five	**Not supported**
Task budgets (beta)	Yes	Yes	**No**	No

Feature	Fable 5	Opus 5	Sonnet 5	Haiku 4.5
Interleaved thinking	Automatic	Automatic	Automatic	**Not supported**
Prior-turn thinking kept	All turns	All turns	All turns	Last turn only
Non-default `temperature`/`top_p`/`top_k`	400	400	400	Allowed
300K batch output beta	No	Yes	Yes	No

Read down any column and the tier ordering breaks somewhere. Read across any row and you will find at least one model that is the exception. That is the point of printing it as a matrix.

One row deserves separate mention because it is a *parameter* deprecation rather than a model one. The documentation lists `temperature`, `top_p` and `top_k` as **deprecated from Claude Opus 4.7 onward**, returning a 400 when set to a non-default value, with the recommended replacement being to omit them and use prompting instead. They remain in the SDK request types so existing code continues to type-check — which is precisely why this fails at runtime rather than at compile time, and precisely why it belongs in a preflight check.

14.9 One Model, Five Front Doors

The same model is reachable through five platforms, addressed differently on each, and — this is the part that catches people — governed by different retirement calendars.

Platform	ID format	Example (Opus 5)	Retirement calendar
Claude API	`claude-{name}-{major}`	`claude-opus-5`	Anthropic
Claude Platform on AWS	**Claude API format**	`claude-opus-5`	**Anthropic**
Microsoft Foundry	—	—	**Anthropic**
Amazon Bedrock	`anthropic.claude-...`	`anthropic.claude-opus-5`	**Bedrock's own**
Google Cloud	Claude API format	`claude-opus-5`	**Google's own**

The deprecation dates published on Anthropic's model-deprecations page apply to **Anthropic-operated platforms**: the Claude API, Claude Platform on AWS, and Microsoft Foundry. **Partner-operated platforms — Amazon Bedrock and Google Cloud — set their own retirement schedules**, so a model's lifecycle status and dates can differ there. If you run on Bedrock and diary the dates from Anthropic's page, you have diaried somebody else's calendar.

Three format details worth committing to memory, because they are exactly the kind of thing that appears as a distractor:

- **Claude Platform on AWS uses Claude API model IDs, not Bedrock-style IDs.** It is on AWS and it does not use `anthropic.` prefixes. It also follows Anthropic's first-party deprecation schedule rather than Bedrock's.
- **Bedrock dropped the `-v1` suffix starting with Claude Sonnet 4.6.** Claude Opus 4.6 (`anthropic.claude-opus-4-6-v1`) is the last Bedrock ID to carry it. Pre-4.6 dated models use the fuller `anthropic.claude-{name}-{major}-{minor}-{YYYYMMDD}-v1:0`.
- **Google Cloud separates the date with `@`:** `claude-haiku-4-5@20251001`. For dateless IDs the Google format simply matches the Claude API.

There is also an endpoint dimension. From **Claude Sonnet 4.5 onward**, Bedrock offers **global endpoints** (dynamic routing for maximum availability) and **regional endpoints** (guaranteed routing through specific geographic regions). Google Cloud offers three types: global, **multi-region** (dynamic routing within a geographic area) and regional. This is a data-residency decision before it is a latency one, and Chapter 61 treats it as such; Chapter 11's `inference_geo` multiplier is the cost side of the same choice.

14.10 The Lifecycle

Four words, with precise meanings.

> DEFINITION
>
> **The model lifecycle**
>
> **Active** — fully supported and recommended for use. **Legacy** — will no longer receive updates and may be deprecated in future. **Deprecated** — still functional but no longer recommended; Anthropic provides a recommended replacement and assigns a retirement date. **Retired** — no longer available; requests to retired models **will fail**.

The documentation adds a warning worth taking at face value: deprecated models are likely to be **less reliable** than active models. Deprecation is not merely a countdown; it is a statement about the support the model is receiving between now and the retirement date.

Notification. Anthropic notifies customers with active deployments of upcoming retirements, providing **at least sixty days' notice before retirement for publicly released models**. Note the shape of the commitment: it is about notice, not about how many people at your organisation read it. The introduction's team received their notice and it went to a departed employee.

Auditing your own usage. The documented procedure is three steps, and it is the single most useful operational habit in this chapter. Go to the Usage page in the Claude Console, click **Export**, and review the CSV, which breaks usage down **by API key and by model**. That tells you what you are actually calling, which is frequently not what you believe you are calling — old batch jobs, a forgotten cron, a service nobody has deployed since the person who wrote it left.

"Not sooner than" is a floor, not a promise. Active models carry tentative retirement dates phrased as "not sooner than". `claude-opus-5` shows a floor of 24 July 2027; `claude-haiku-4-5-20251001` shows 15 October 2026, which is under three months from the time of writing. A floor is the earliest the door can close. It is not a commitment that the door stays open.

14.11 Deprecation Has Costs, and Anthropic Says So

A short section, included because it is unusual and because the exam blueprint's responsible-AI thread runs through it.

The documentation states plainly that models are deprecated and retired to ensure capacity for new releases, and then lists the downsides rather than eliding them: users who value specific models must migrate; **researchers lose access to models for ongoing and comparative studies**; and model retirement introduces **safety- and model welfare-related risks**. Anthropic states a hope to make past models publicly available again at some point, and has committed to **long-term preservation of model weights** and other mitigating measures, documented under *Commitments on Model Deprecation and Preservation*.

Two reasons this belongs in a technical chapter. The reproducibility point is directly operational: if your compliance regime requires that a decision made in March 2026 can be re-derived, and the model that made it is retired, "re-run it" is not available to you. What is available is logging the inputs, the model ID, the parameters and the outputs at the time — which Chapter 57 builds and which no amount of weight preservation substitutes for. And the welfare point is a genuine position that a certified architect should be able to state accurately, whatever they think of it, rather than being surprised by it in a stakeholder conversation.

14.12 Asking the Platform Instead of a Table

Every table in this chapter is a snapshot of something that changes. The platform offers a live alternative.

The **Models API** returns the models available to your account. The documentation states that the response includes `max_input_tokens`, `max_tokens`, and a `capabilities` **object** for every available model. That is exactly the data this chapter has been tabulating by hand, available programmatically and current by construction. Chapter 26 covers the endpoint in full alongside the rest of the API surface.

The right architecture uses both, and the mini project is built that way:

- **A checked-in registry** is your source of truth for the facts you have verified, with provenance and an explicit UNKNOWN. It works offline, it is reviewable in a pull request, and it is what your tests assert against.
- **The Models API** is your reconciliation. Fetch it in CI, diff it against the registry, and fail the build when they disagree.

The failure mode of registry-only is silent staleness. The failure mode of API-only is that you cannot test offline, cannot review a change, and have no record of what you believed last month. Neither is sufficient; the diff between them is the alert you actually want.

14.13 Visual Analogies

VISUAL ANALOGY

A parts catalogue with expiry dates

A model ID is a part number. It specifies a manufactured item with fixed properties, and the manufacturer publishes a catalogue giving each part's dimensions, price, tolerances and end-of-life date. Ordering by part number gets you the same part every time. Ordering by description — "the fast one" — gets you whatever is currently called that.

Where it breaks. Physical parts do not change behaviour while the part number stays the same, and models do: section 14.4's serving infrastructure moves under a pinned ID. The tolerances in this catalogue are guaranteed on the weights and not on the observable output. It also breaks on discontinuation. A discontinued bolt sits in a warehouse for a decade; a retired model returns an error the following morning.

VISUAL ANALOGY

Semantic versioning, half-adopted

Developers read `claude-sonnet-5` as a version number and reach for semantic-versioning instincts: majors break, minors add, and something somewhere resolves to "latest compatible". Two of those instincts are right — a generation change does break things, as section 14.19's eleven-row diff shows — and the third is exactly wrong. There is no resolution step. The dateless ID is the pin.

Where it breaks. Semantic versioning is a promise made by the publisher about compatibility, machine-checkable by a dependency resolver. There is no such promise here and no resolver. Nothing in `claude-sonnet-5` tells you that `temperature` now 400s; only the documentation and your own preflight do. Treat the ID as an opaque key into a catalogue, not as a version string you can reason about.

VISUAL ANALOGY

A hire fleet, not a purchase

You are not buying a model; you are hiring capacity from a fleet that the operator refreshes on their schedule. The vehicle you booked will be there, exactly as specified, until the day it is retired from the fleet — and you get sixty days' notice, sent to the email address on the account.

Where it breaks. A hire company will not usually retire the whole class of vehicle you depend on with no replacement of identical characteristics, and here the replacement is genuinely different: different thinking mode, different accepted parameters, different price. The analogy also understates your obligations. A hire car needs no integration work; a model migration is section 14.19's checklist, and it is your work, not the operator's.

14.14 Architecture: The String Is the Bug

Almost every problem in this chapter traces to one design decision that nobody makes deliberately: the model is a string literal, scattered.

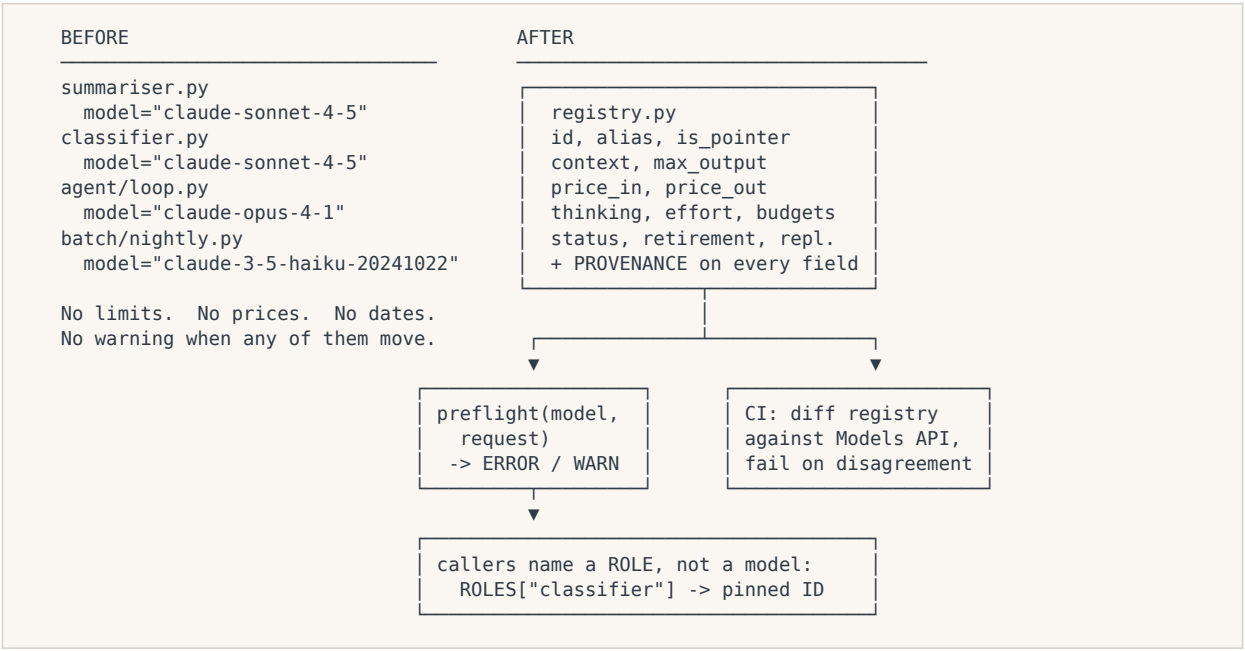


Figure 14.1 — From a scattered string to a registry with provenance.

Four properties of the right-hand side, each of which removes a failure from the introduction.

One place to change. A deprecation becomes a one-line edit and a test run, not a repository-wide search whose false negatives are invisible.

Facts travel with the name. Code that needs the output ceiling asks the registry rather than a constant defined nearby and copied twice.

Provenance is explicit. Every field is either verified on a stated date or **UNKNOWN**, and UNKNOWN raises rather than defaulting. This is the discipline of Chapter 12 — "not established" is a permitted answer — applied to configuration. A registry that quietly guesses the context window of a model nobody checked is worse than no registry, because you will trust it.

Callers name a role, not a model. `ROLES["classifier"]` rather than `"claude-haiku-4-5-20251001"`. Chapter 15 builds the selection procedure that populates that map; the indirection is what makes the procedure's output actionable rather than advisory.

14.15 Examples: Four Strings, Four Verdicts

Concrete readings, to make the rules bite before the code.

`claude-sonnet-4-5`**.** A pre-4.6 convenience alias that resolves to `claude-sonnet-4-5-20250929`. Usable, and the snapshot it points at has a retirement floor of 29 September 2026 — under two months from the time of writing. **Verdict: pin the dated ID, and diary the date.**

`claude-sonnet-5`**.** A dateless 4.6-generation ID, which means it is already a pinned snapshot. There is nothing to pin further. **Verdict: correct as written; the work is elsewhere, in the eleven behavioural differences of section 14.19.**

`claude-opus-4-1`**.** An alias to `claude-opus-4-1-20250805`, which is **deprecated with a retirement date of 5 August 2026**. Seven days from the time of writing. **Verdict: this is an incident, not a backlog item.**

Recommended replacement: `claude-opus-4-8`.

`claude-sonnet-4.5`**.** Not a model string at all — dots rather than hyphens. It resolves to nothing, and the mini project's `resolve()` raises on it. Worth including because it is the single most common typo, it is caught for free by a registry and never by a string literal, and in production it costs you a round trip and an unhelpful error at exactly the wrong moment.

14.16 Code Walkthrough: A Registry That Refuses to Guess

The mini project is four modules and forty-one tests. Its most important design decision is not a feature; it is a refusal.

Provenance is a first-class field. Every value in the registry is either verified against the official documentation on a stated date, or it is `UNKNOWN`. There is no third state, and specifically there is no "probably the same as the previous model" — which is the assumption that produced all four of the introduction's failures.

```
UNKNOWN = "UNKNOWN"
class Unverified(Exception):
    """Raised when code asks for a fact the registry does not hold."""
@dataclass(frozen=True)
class Model:
    id: str
    family: str
    context: object = UNKNOWN
    max_output: object = UNKNOWN
    price_in: object = UNKNOWN
    ...
    def get(self, name):
        v = getattr(self, name)
        if v is UNKNOWN or v == UNKNOWN:
            raise Unverified(
                f"{self.id}: '{name}' is not verified in this registry. "
                f"Check platform.claude.com/docs before relying on it.")
        return v
    def known(self, name):
        return getattr(self, name) != UNKNOWN
```

`get()` raises; it does not default. That one line is the difference between a registry that tells you what you know and a registry that tells you what it has made up. The current four models are fully populated because every field was checked; older models carry only what was verified, and the pricing helper in section 14.21 raises `Unverified` rather than pricing a model whose rate card we did not confirm. There is a test asserting exactly that, and it is the test that best captures the book's disposition:

```
def test_cost_refuses_unpriced_models():
    with pytest.raises(Unverified):
        mg.cost("claude-opus-4-1", 1000, 100)
```

Aliases are modelled as pointers, and only where they exist.

```
ALIASES = {m.alias: m.id for m in MODELS.values()
           if m.alias and m.alias_is_pointer}
def resolve(name):
```

```
    if name in MODELS:
        return name
    if name in ALIASES:
        return ALIASES[name]
    raise KeyError(f"unknown model string: {name!r}")
```

The comprehension filters on `alias_is_pointer`, which is `True` only for pre-4.6 models. A 4.6-generation dateless ID such as `claude-sonnet-5` never enters the alias map, because it is not an alias — it is the snapshot. Section 14.3's misconception is therefore unrepresentable in this code, and a two-line test keeps it that way:

```
def test_dateless_46_generation_ids_are_snapshots_not_aliases():
    assert "claude-sonnet-5" not in reg.ALIASES
    assert reg.is_pinned("claude-sonnet-5")
```

`preflight()` **reproduces documented 400s, not guessed ones.** Each check maps to a sentence in the platform documentation, and each returns a `Finding` at one of three severities: `ERROR` (the request will fail, or the model is retired), `WARN` (it will succeed and bite you later), `NOTE` (a fact the registry does not hold). The conditional-disable rule on Opus 5 is representative:

```
if (ttype == "disabled" and m.can_disable == "effort<=high"
        and effort in ("xhigh", "max")):
    out.append(Finding(
        ERROR, "disable-at-high-effort",
        f"{model_id} returns 400 for thinking 'disabled' at "
        f"effort {effort!r}"))
```

`can_disable` is deliberately not a boolean. Fable 5 is `False`, Sonnet 5 is `True`, and Opus 5 is the string `"effort<=high"`, because the documented behaviour is genuinely three-valued. Squashing it into a boolean would have made the registry wrong in a way no test could catch, since both wrong answers are plausible.

Severity escalates with the calendar. A deprecation more than thirty days out is a `WARN`; inside thirty days it becomes an `ERROR` that fails a build.

```
if m.status == "deprecated":
    days = (m.retirement - today).days
    out.append(Finding(
        ERROR if days <= 30 else WARN, "deprecated",
        f"{model_id} is deprecated and retires {m.retirement} "
        f"({days} days). Replacement: {m.replacement}"))
```

`today` is a parameter rather than a call to `date.today()`, which is what makes the behaviour testable at all — the suite asserts `WARN` on 10 June 2026 and `ERROR` on 29 July for the same model. A time-dependent rule that cannot be tested at a chosen time is a rule that will be discovered in production.

14.17 Mini Project: The Model Registry and Preflight

Goal. Replace every model string literal in a codebase with a record that carries its own limits, prices, capabilities and expiry date; then validate outbound requests against it before they cost a round trip.

14.17.1 Architecture

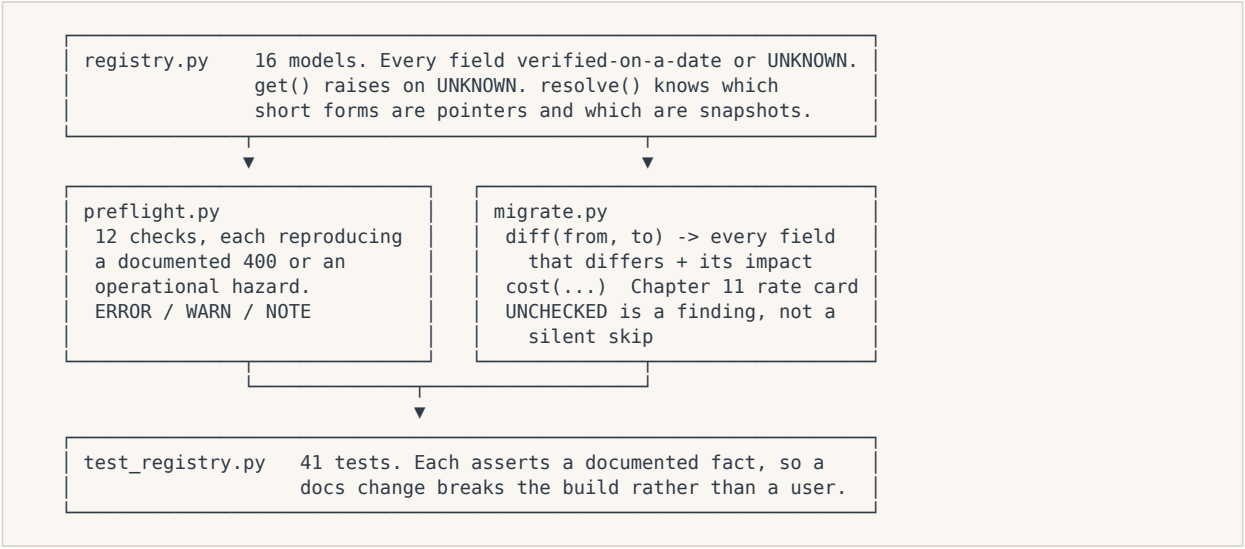


Figure 14.2 — Four modules, one job: make the model string informative.

14.17.2 Folder structure

```
ch14/
  registry.py         the catalogue, with provenance
  preflight.py        request validation against a model
  migrate.py          migration diff and the cost model
  report.py           the five panels below
  test_registry.py    41 tests, 0.11 seconds
```

No API key is needed and no network call is made. That is deliberate: this is the layer that must work when the network does not, and it is the layer a reviewer should be able to read in a pull request.

14.17.3 Panel A — the family as a table

```
A  THE CURRENT FAMILY (verified 2026-07-29)
                     model       ctx      out    in$   out$  thinking  effort  tb
            claude-fable-5 1,000,000  128,000     10     50  adaptive     yes   y
             claude-opus-5 1,000,000  128,000      5     25  adaptive     yes   y
           claude-sonnet-5 1,000,000  128,000      2     10  adaptive     yes   n
 claude-haiku-4-5-20251001   200,000   64,000      1      5  extended      no   n
```

Read the last three columns rather than the first three. Sonnet 5 has adaptive thinking and all five effort levels and **no task budgets**; Haiku 4.5 has extended thinking and **no effort parameter at all**. If tiers were a ladder, neither row could exist.

14.17.4 Panel B — the lifecycle clock

```
B  THE LIFECYCLE CLOCK, as at 2026-07-29
                     model      status      retires   days  replacement
claude-3-7-sonnet-20250219     retired  2026-02-19   -160  sonnet-4-6
 claude-3-5-haiku-20241022     retired  2026-02-19   -160  haiku-4-5-20251001
   claude-3-haiku-20240307     retired  2026-04-20   -100  haiku-4-5-20251001
    claude-opus-4-20250514     retired  2026-06-15    -44  opus-4-8
  claude-sonnet-4-20250514     retired  2026-06-15    -44  sonnet-4-6
  claude-opus-4-1-20250805  deprecated  2026-08-05      7  opus-4-8
```

```
claude-sonnet-4-5-20250929      active ~2026-09-29     62
 claude-haiku-4-5-20251001      active ~2026-10-15     78
  claude-opus-4-5-20251101      active ~2026-11-24    118
           claude-opus-4-6      active ~2027-02-05    191
         claude-sonnet-4-6      active ~2027-02-17    203
           claude-opus-4-7      active ~2027-04-16    261
           claude-opus-4-8      active ~2027-05-28    303
            claude-fable-5      active ~2027-06-09    315
           claude-sonnet-5      active ~2027-06-30    336
             claude-opus-5      active ~2027-07-24    360
```

The `~` marks a "not sooner than" floor rather than a commitment. Three observations you would not get from prose.

Five models retired inside the last six months. Sonnet 3.7 and Haiku 3.5 in February, Haiku 3 in April, Sonnet 4 and Opus 4 in June. That is the actual cadence, and it is faster than most release calendars assume.

One model is seven days from retirement, and it is the one in the introduction. A CI job that printed this table would have produced a `days` column reading `7` in a build log every day since 5 June.

Two active models have floors inside three months. `claude-sonnet-4-5-20250929` at sixty-two days and `claude-haiku-4-5-20251001` at seventy-eight. Both are perfectly supported today. Both are closer to their earliest possible retirement than a typical migration project is long, which is a scheduling fact, not a warning.

14.18 Panel C — preflight over requests a real team would send

Nine requests, each of which somebody has actually sent. Abbreviated to the verdicts; `report.py` prints the full messages.

```
  [claude-opus-5]  carried-over thinking config  -> ERROR
    ERROR thinking-enabled-unsupported
          thinking.type "enabled" returns 400 on claude-opus-5; use
          {"type": "adaptive"} with output_config.effort
  [claude-sonnet-5]  carried-over temperature  -> ERROR
    ERROR sampling-locked
          non-default temperature returns 400 on claude-sonnet-5 on
          every request, thinking or not
  [claude-haiku-4-5]  adaptive on Haiku  -> ERROR
    WARN  alias-not-pinned
          'claude-haiku-4-5' is a convenience alias resolving to
          'claude-haiku-4-5-20251001'. It can move under you.
    ERROR thinking-adaptive-unsupported
          thinking.type "adaptive" returns 400 on
          claude-haiku-4-5-20251001; use {"type": "enabled", ...}
  [claude-opus-5]  effort=adaptive  -> ERROR
    ERROR effort-adaptive
          'adaptive' is a thinking mode, not an effort level
  [claude-sonnet-5]  task budget, wrong model  -> ERROR
    ERROR task-budget-unsupported
          claude-sonnet-5 does not support task budgets
    ERROR task-budget-too-small
          task_budget.total=8,000 is below the 20,000 minimum
  [claude-opus-5]  prefill on a 5-series model  -> ERROR
    ERROR prefill-thinking-default-on
```

```
         claude-opus-5 has thinking ON by default, so this prefill
         fails unless you set thinking 'disabled'
  [claude-opus-4-1]  deprecated model, one week left  -> ERROR
    WARN  alias-not-pinned
    ERROR deprecated
         claude-opus-4-1-20250805 is deprecated and retires
         2026-08-05 (7 days). Replacement: claude-opus-4-8
    NOTE  unverified   'context', 'price_in', 'max_output'
  [claude-haiku-4-5]  128k output on Haiku  -> ERROR
    ERROR max-tokens-over-ceiling
         max_tokens=128,000 exceeds claude-haiku-4-5-20251001's
         64,000-token output limit
  [claude-opus-5]  a request that is actually fine  -> OK
```

Every one of these fails at runtime, on a live request, having consumed a round trip — and several of them fail *intermittently*, on the subset of traffic that happens to set the offending parameter, which is the worst possible failure shape. The last case exists to prove the validator can say yes; a checker that never passes anything is not a checker.

Notice the three `NOTE` findings on the deprecated model. The registry does not hold Opus 4.1's context window, price or output ceiling, so it says so rather than inventing them. That is the honest behaviour and it is also the useful one: it tells you precisely which facts you must go and look up before you can finish the migration.

14.19 Panel D — the one-line migration, expanded

The introduction's engineer changed eleven characters. Here is what the registry says they changed.

```
claude-sonnet-4-5  ->  claude-sonnet-5   (11 differences)
                        field       from         to  impact
                thinking mode   extended   adaptive  code change: thinking config
          can disable thinking  UNCHECKED  UNCHECKED  verify
        thinking on by default         no        yes  cost change: thinking now on
                 effort levels       none      all 5  re-sweep effort on your evals
        sampling params locked         no        yes  code change: remove temperature
     prior-turn thinking kept       last        all  context growth changes
         interleaved thinking     header       auto  tool-loop reasoning changes
            max output tokens     64,000    128,000  limits changed
               context window  UNCHECKED  UNCHECKED  verify
     thinking display default summarized    omitted  thinking text visibility
       300k batch output beta         no        yes  review
```

Eleven differences, of which two require code changes before the first request succeeds (thinking mode, sampling parameters), **one changes the bill immediately** (thinking on by default), **one changes how fast context grows in a long session** (prior-turn thinking now kept rather than stripped), and **one is an evaluation task rather than a code task** (effort levels newly available, and Chapter 13's rule that settings do not transfer between models).

Two of the eleven are `UNCHECKED`, and that is a deliberate output rather than a gap. `diff()` reports fields it cannot compare instead of silently skipping them, because **a migration you cannot check is a finding, not an absence of one.** The alternative — omitting unverifiable rows — produces a diff that looks shorter and cleaner and is a lie.

The second diff in the report is the migration the deprecation notice actually recommends, `claude-opus-4-1` to `claude-opus-4-8`, and it comes out at twelve differences with six `UNCHECKED`. The lesson is the same in both directions: **the recommended replacement is a recommendation about capability, not a statement of compatibility.** Anthropic's page tells you where to go. It does not tell you that your request builder will still work when you get there.

14.20 Panel E — one workload, four models

```
E  ONE WORKLOAD, FOUR MODELS
  1,000,000 requests/month, 4,000 in / 600 out, no caching
                       model    monthly $   vs Haiku       +batch
              claude-fable-5       70,000      10.0x       35,000
               claude-opus-5       35,000       5.0x       17,500
             claude-sonnet-5       14,000       2.0x        7,000
   claude-haiku-4-5-20251001        7,000       1.0x        3,500
  Sonnet 5 introductory $14,000/month becomes $21,000 on 2026-09-01
  (1.5x, already on the calendar).
  Opus 5 with an 8,000-token thinking trace: $35,000 -> $235,000 (6.7x)
```

Three readings, and the third is the one that changes decisions.

The family spans exactly ten-fold, and the Batch API halves every row. A workload that tolerates a delay can run Fable 5 for less than Opus 5 costs synchronously, which reframes "we cannot afford the top model" as a question about latency tolerance rather than budget.

The scheduled price rise is larger than most optimisations. Sonnet 5 going from $14,000 to $21,000 on 1 September is a $7,000-a-month change that arrives without a deployment. A team that spent a sprint shaving 10% off their prompt would have saved $1,400 and lost $7,000 on the same page of the ledger.

Thinking dominates everything else on this page. Opus 5 at $35,000 becomes $235,000 with an 8,000-token trace — 6.7×, larger than the entire spread from Haiku to Fable. That is Chapter 13's arithmetic landing on Chapter 14's rate card, and it is why model selection and effort selection are one decision rather than two. Chapter 15 makes that decision properly.

14.21 Evaluation and deployment

Run `python3 report.py` (instant, no network) and `python3 -m pytest` (41 tests, 0.11 seconds). To adopt the pattern:

1. **Grep for model strings.** Every literal is a row you are missing. Move each into `registry.py`.
2. **Add a `ROLES` map** — `{"classifier": "claude-haiku-4-5-20251001", "agent": "claude-opus-5"}` — and have callers name a role. Chapter 15 populates it with measurements rather than intuitions.
3. **Call `preflight()` in your request builder** in development and in CI, on every request shape your code can emit. In production, log its findings rather than blocking; a `WARN` you have chosen to accept should not page anyone.

4. **Add a scheduled job** that fetches the Models API, diffs it against the registry, and fails on disagreement. That is the alert that would have fired on 5 June.
5. **Export the Console usage CSV monthly** and diff the model column against your registry's active set. This catches the services nobody remembers deploying, which is where retired-model incidents actually live.

14.22 Hands-on Labs

Lab 1 — Inventory. Find every model string in a codebase you own, including configuration files, notebooks and infrastructure-as-code. *Success criterion:* a list with a file and line for each, and a count of distinct model strings — which will be larger than you expect.

Lab 2 — Audit against the clock. For each distinct string, look up its status and retirement date on the model-deprecations page. *Success criterion:* every string classified active, deprecated or retired, with a date, and any deprecated one raised as a ticket with an owner.

Lab 3 — Reconcile with reality. Export the Console usage CSV and compare the model column against Lab 1's list. *Success criterion:* you can explain every model that appears in the CSV and does not appear in your code, and every one that appears in your code and not the CSV.

Lab 4 — Preflight your real request shapes. Feed each request shape your code can produce into `preflight()`. *Success criterion:* zero ERRORs, and every WARN either fixed or explicitly accepted in writing.

Lab 5 — Price the calendar. Build a twelve-month cost projection for your current model mix that includes the Sonnet 5 price change on 1 September 2026 and the retirement floor of every model you use. *Success criterion:* a chart with steps in it, and a named owner for each step.

14.23 Lab Solutions

Lab 1. `grep -rn "claude-" --include='*.py' --include='*.ts' --include='*.yaml' .` finds most of it. The ones it misses are the expensive ones: strings assembled from parts, values in a database, defaults in a deployment template. Search your secrets and config stores too.

Lab 2. The deprecation page is the authority and it changes; check it rather than reciting section 14.17.4's table, which was true on 29 July 2026 and will not be true when you read this. Anything **deprecated** is an incident with a date, not a backlog item — the introduction's team had seven days, and seven days is not enough for a migration that turns out to have eleven differences in it.

Lab 3. The two directions fail differently. In the CSV and not in the code means something is calling the API that you do not control or have forgotten: an old batch job, a notebook on someone's laptop, a service whose repository was archived. In the code and not the CSV means dead code, or a path that has never executed in production — which is worth knowing before you rely on it.

Lab 4. Enumerate request *shapes*, not requests: the distinct combinations of thinking configuration, effort, sampling parameters, `max_tokens` and tool choice that your builder can produce. A builder with three optional flags has eight shapes and you have probably tested two.

Lab 5. The steps come from three sources: the Sonnet 5 introductory pricing ending on 31 August 2026, the retirement floors of the models you use, and any planned migration. The point of the chart is that the steps have owners; an unowned step is a surprise with a date attached.

14.24 Best Practices

BEST PRACTICE

Working with the model family

1. **Pin the snapshot.** Use `claude-haiku-4-5-20251001`, not `claude-haiku-4-5`. For 4.6-generation dateless IDs there is nothing further to pin.
2. **Never write a model string twice.** One registry, one place to change, callers naming roles.
3. **Carry provenance.** Every fact is verified on a date or explicitly unknown. Unknown must raise, not default.
4. **Diary the retirement dates**, including the "not sooner than" floors of active models. A floor is not a promise.
5. **Diff the registry against the Models API in CI.** The API is current; the registry is reviewable. The disagreement is the alert.
6. **Export the Console usage CSV monthly.** It is the only view of what you are actually calling.
7. **Preflight every request shape** in CI. Twelve cheap checks beat one 400 in production.
8. **Read the matrix, not the tier.** Sonnet 5 has no task budgets; Haiku 4.5 has no effort parameter.
9. **Treat a recommended replacement as capability advice, not compatibility advice.** Run the diff.
10. **Re-sweep effort and prompts after any model change.** Chapter 13's rule: settings do not transfer.
11. **Put the price calendar in the cost model.** Sonnet 5's rise on 1 September is bigger than most optimisations.
12. **Know which platform's calendar governs you.** Bedrock and Google Cloud set their own; Claude Platform on AWS and Microsoft Foundry follow Anthropic's.
13. **Log the model ID with every response** you store. When the model is retired, the log is the only reproducibility you have.
14. **Keep a regression suite pinned to a model ID.** It is the only instrument that detects serving-infrastructure drift.

14.25 Common Mistakes

COMMON MISTAKE

Believing a dateless ID resolves to "latest"

Symptom. A team expects `claude-opus-5` to improve over time, and is surprised both that it does not and that a newer model shipped without their noticing. **Cause.** Reading the 4.6-generation dateless format as an evergreen alias. It is a pinned snapshot; only pre-4.6 short forms are pointers. **Fix.** Treat every ID as opaque and pinned. Subscribe to the release notes; put a model review on a calendar. **Detection.** Ask what your system would do if a better model launched tomorrow. If the answer is "pick it up automatically", the mental model is wrong.

COMMON MISTAKE

Assuming the tier ordering implies feature support

Symptom. A 400 on a subset of traffic after routing some requests to a different tier — task budgets on Sonnet 5, `effort` on Haiku 4.5, interleaved thinking on Haiku. **Cause.** Reading Fable → Opus → Sonnet → Haiku as nested capability. Feature support lists do not nest. **Fix.** Encode the matrix and validate against it. Section 14.18's preflight catches all three of those. **Detection.** A 400 rate that correlates with a routing decision rather than with input content.

COMMON MISTAKE

Carrying request parameters across a model change

Symptom. Every request 400s immediately after a model bump, on `temperature`, on `thinking.type`, or on a prefilled assistant message. **Cause.** `temperature`, `top_p` and `top_k` are deprecated from Opus 4.7 onward and return 400 on non-default values; `thinking.type: "enabled"` is rejected on 4.7 and later; thinking defaults to on across the 5-series, and you cannot prefill while thinking is on. The parameters remain in the SDK request types, so nothing fails to type-check. **Fix.** Run the migration diff before the migration. Preflight the request shapes. **Detection.** A 100% error rate immediately after a deploy whose diff is one string.

COMMON MISTAKE

Diarying the wrong platform's retirement calendar

Symptom. A model retires on a partner platform on a date that does not appear on Anthropic's deprecation page, or survives past a date that does. **Cause.** Anthropic's dates govern Anthropic-operated platforms — the Claude API, Claude Platform on AWS, Microsoft Foundry. Amazon Bedrock and Google Cloud set their own schedules. **Fix.** Diary the calendar of the platform you actually call. If you run on more than one, diary both. **Detection.** Check whether your runbook cites Anthropic's page while your traffic goes to a `anthropic.claude-` Bedrock ID.

COMMON MISTAKE

Sending the deprecation email to a departed employee

Symptom. A retirement arrives unannounced despite the documented sixty days' notice. **Cause.** The commitment is about notice being sent, not about it being read. Account-owner email addresses rot. **Fix.** Point the account contact at a monitored distribution list, and back it with a CI job that reads the deprecation status rather than an inbox. **Detection.** Ask who received the last deprecation email. If nobody can name them, this mistake is live in your organisation right now.

COMMON MISTAKE

Hard-coding limits next to the model string

Symptom. `max_tokens=128000` on a model whose ceiling is 64,000, or a chunking routine sized for 1M context pointed at a 200K model. **Cause.** Limits copied into constants near the call site, then the model changed and the constants did not. **Fix.** Read limits from the registry. `preflight()` compares `max_tokens` against the model's ceiling and knows about the 300K batch beta and which models support it. **Detection.** Grep for numeric literals near model strings. Every one is a stale fact waiting to happen.

COMMON MISTAKE

Treating "not sooner than" as a support commitment

Symptom. A migration scheduled for the month a floor expires, with no slack. **Cause.** Reading a floor as a guarantee. It is the earliest the door can close, not the latest. **Fix.** Plan migrations against the floor minus your actual migration time, which section 14.19 suggests is longer than a string change. **Detection.** Compare your migration lead time against the shortest floor in your registry. `claude-sonnet-4-5-20250929` sat at sixty-two days at the time of writing.

14.26 Performance Considerations

The family's latency ordering is documented rather than measured here: Fable 5 slower, Opus 5 moderate, Sonnet 5 fast, Haiku 4.5 fastest. Chapter 11's two-term model explains why that ordering is coarse and why it can invert. Latency is `prefill + output_tokens × per_token_decode`, and decode dominates at 82–99% of wall clock. A faster model that thinks at high effort can be slower end-to-end than a slower model at low effort, because the token count moved and the per-token rate did not move as much. **Tier ordering is a per-token statement; your users experience a per-request one.**

Three consequences worth designing around. Haiku 4.5's 200K context is a fifth of the others', so a retrieval pipeline sized for 1M does not merely cost more on a larger model — it fails on Haiku, and Chapter 52's context budget is where that gets planned rather than discovered. The 300K batch output beta is unavailable on both Fable 5 and Haiku 4.5, so a long-generation batch workload has exactly four eligible models. And the endpoint choice on Bedrock and Google Cloud — global versus regional versus multi-region — is a latency and availability decision that Anthropic's own documentation frames primarily in terms of data routing; treat the residency requirement as the constraint and the latency as the consequence.

14.27 Security Considerations

The model ID is a supply-chain dependency. It names an artefact you did not build, running on infrastructure you do not control, whose behaviour can shift under a stable identifier. Treat it as you would any pinned dependency: pin it, record it, review changes to it, and keep a test suite that would notice if it moved. Chapter 58 covers the injection surface; this is the supply-chain half.

Log the model ID with every stored output. When a model is retired, the weights are preserved by Anthropic but not available to you, so a decision you cannot re-run is a decision you can only defend from your own records. Store the ID, the parameters, the input and the output. Chapter 57 builds this properly

and Chapter 61 covers the retention obligations that come with it.

A model change is a change to your safety posture. Safety classifiers are part of the serving infrastructure, not the weights, and different models have different refusal behaviour and different documented refusal categories — Fable 5's `reasoning_extraction` category from Chapter 13 has no analogue on Haiku 4.5. Any migration should re-run your adversarial suite, not only your quality suite.

Data residency travels with the platform, not the model. The same model on a Bedrock regional endpoint, a Google Cloud multi-region endpoint and the Claude API with `inference_geo: "us"` gives three different answers to "where was this processed", at three different prices. If a contract constrains the answer, the model ID is the least important part of the configuration.

Project Glasswing is a reminder that availability is a policy. Mythos 5 exists, is documented, is priced, and you cannot call it. Architectures should not assume that a model appearing in a comparison table is a model you can procure.

14.28 Real Production Examples

The mixed fleet. Most serious systems run three or four models at once: Haiku 4.5 for classification and routing, Sonnet 5 for the main path, Opus 5 for the agentic work, and a batch tier for anything that can wait. The registry pattern is what makes that manageable, because "which model does the router use" becomes a lookup rather than an archaeology exercise. Chapter 50 covers the orchestrator–subagent split where this is the standard shape.

The regulated deployment. A financial or healthcare workload typically pins to a dated or dateless snapshot, runs on a regional endpoint for residency, logs the model ID with every decision, and treats a model migration as a change requiring re-validation. The retirement floor becomes a compliance date rather than an engineering one, and sixty days' notice is short against a validation cycle — which is the argument for tracking floors on active models rather than waiting for deprecation.

The cost-driven downgrade. A team on Opus 5 measures Sonnet 5 on their own evaluation set, finds the difference not established at their sample size, and moves — halving the bill. Two things make it work: Chapter 8's intervals, so "not established" is a real finding rather than wishful thinking, and the registry, so the change is one line. Two things nearly break it: task budgets, which Sonnet 5 does not support, and the price rise on 1 September, which halves the saving.

The forgotten batch job. The commonest retired-model incident is not a service; it is a nightly job on a model nobody has thought about since 2024. It fails at 03:00, alerts to a channel nobody watches, and is discovered by a downstream data gap a week later. The Console usage CSV is the only reliable way to find these before they find you.

14.29 Certification Notes

Concept	Exam domain	What is likely to be asked
Model IDs and pinning	D5 Context Management & Reliability	Dateless 4.6+ IDs are snapshots, not aliases; pre-4.6 short forms are pointers

Concept	Exam domain	What is likely to be asked
Weights vs serving infrastructure	D5	Pinning fixes weights, not behaviour; infra is the likeliest drift cause
Context and output limits	D5, D4	1M/128K on the 5-series; 200K/64K on Haiku 4.5; 300K batch beta and who supports it
Reliable vs training cutoff	D5	Two distinct dates; they differ on Haiku 4.5
Feature matrix	D1, D4	Sonnet 5 has no task budgets; Haiku 4.5 has no `effort` and no interleaved thinking
Parameter deprecation	D4	`temperature`/`top_p`/`top_k` 400 from Opus 4.7 onward; still in the SDK types
Platform IDs	D1	Bedrock `anthropic.` prefix; Google `@` dates; Claude Platform on AWS uses Claude API IDs
Retirement calendars	D5	Anthropic's dates govern Anthropic-operated platforms only
Lifecycle vocabulary	D5	Active / legacy / deprecated / retired; ≥60 days' notice
Auditing usage	D5	Console → Usage → Export → CSV by API key and model
Models API	D5	`max_input_tokens`, `max_tokens`, `capabilities` per model
Glasswing	D1	Mythos 5 and Mythos Preview: invitation-only, defensive cybersecurity

CERTIFICATION NOTE

Exam Tips

- **"Which model would you start with?"** The documentation says **Claude Opus 5** for complex agentic coding and enterprise work, and **Claude Fable 5** where the highest capability is needed.
- **A dateless ID is a pinned snapshot.** Any option describing `claude-sonnet-5` as resolving to the latest version is wrong.
- **Haiku 4.5 is the odd one out on almost every row**: extended thinking only, no `effort`, no interleaved thinking, 200K/64K, and the only model whose two cutoffs differ.
- **Sonnet 5 supports every effort level and no task budgets.** A favourite because it breaks the tier intuition.
- **Output is exactly 5× input** on every current model. Useful for arithmetic questions.
- **Sixty days is the notice commitment**, not the deprecation window.
- **Bedrock and Google Cloud set their own retirement schedules.** Claude Platform on AWS does not — it follows Anthropic's.
- **If a question describes drift on an unchanged model ID**, the answer is serving infrastructure, not weights.

14.30 Twenty Practice Questions

1. Which model does the documentation suggest starting with when you are unsure?
2. What is the context window and max output of Claude Haiku 4.5?
3. Does `claude-sonnet-5` resolve to the latest Sonnet model? Explain.
4. Which pre-4.6 short forms are convenience pointers, and what do they point at?
5. A pinned model ID starts behaving differently. What changed?
6. What is the difference between the reliable knowledge cutoff and the training data cutoff?
7. Which current models have different values for those two dates?
8. Which current model does **not** support the `effort` parameter?
9. Which current model does **not** support task budgets, despite supporting adaptive thinking?
10. Which models support the 300K batch output beta?
11. What is Claude Sonnet 5's listed price, and what is it actually charging until when?
12. What is the ratio of output to input pricing across the current family?
13. How is Claude Opus 5 addressed on Amazon Bedrock? On Google Cloud?
14. Which model ID format does Claude Platform on AWS use, and whose deprecation schedule applies?
15. Name the four lifecycle states and what each means for a running system.
16. How much notice does Anthropic commit to before retiring a publicly released model?
17. How do you find out which models your organisation is actually calling?
18. What does "not sooner than 2027-07-24" guarantee?
19. Which fields does the Models API return for each model?
20. What are Claude Mythos 5 and Claude Mythos Preview, and can you use them?

14.31 Answers and Explanations

1. **Claude Opus 5**, for complex agentic coding and enterprise work; **Claude Fable 5** where you need the highest available capability. *Distractors usually offer Sonnet 5 as "the balanced default" — reasonable engineering, but not what the documentation says.*
2. **200K context, 64K max output.** It is the only current model at those figures; the other three are 1M and 128K. *Answering 1M/128K for the whole family is the trap.*
3. **No.** From the 4.6 generation onward, dateless IDs are the canonical, pinned snapshot for that release. Anthropic does not update the weights of an existing ID; a new version ships under a new ID. *The documentation flags this as a common misconception.*
4. **Pre-4.6 short forms** such as `claude-sonnet-4-5` and `claude-haiku-4-5` point at the most recent **dated snapshot** for that minor version — `claude-sonnet-4-5-20250929`, `claude-haiku-4-5-20251001`. *These are the only true aliases.*
5. **The serving infrastructure** — request router, safety classifiers, sampling logic. Weights are fixed for a given ID; the documentation names an infrastructure update as the most likely cause of unexpected behavioural differences on a stable ID. *"The weights were updated" is explicitly ruled out.*

6. The **reliable knowledge cutoff** is the date through which knowledge is most extensive and reliable; the **training data cutoff** is the broader date range of training data used. *They are not synonyms and the docs distinguish them deliberately.*
7. **Claude Haiku 4.5**: reliable cutoff February 2025, training cutoff July 2025. The other three current models have identical values for both. *A five-month gap on the cheapest model.*
8. **Claude Haiku 4.5.** It supports neither `effort` nor interleaved thinking, and uses extended thinking rather than adaptive. *This is three separate gotchas on one model.*
9. **Claude Sonnet 5.** It supports adaptive thinking and all five effort levels; task budgets are beta on Opus 5, Fable 5, Mythos 5, Opus 4.8 and Opus 4.7 only. *The cleanest counterexample to tier ordering.*
10. **Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6**, via the `output-300k-2026-03-24` beta header on the Message Batches API. *Neither Fable 5 nor Haiku 4.5 is on the list.*
11. Listed at **$3 / $15 per MTok**; **introductory pricing of $2 / $10 applies through 31 August 2026**. *A 50% increase already scheduled — which is the point of the question.*
12. **Exactly 5×** on every current model: $1/$5, $2/$10, $5/$25, $10/$50. *This is why thinking tokens, billed as output, dominate a cost model.*
13. Bedrock: `anthropic.claude-opus-5`. Google Cloud: `claude-opus-5`, matching the Claude API format. *Note the `-v1` suffix was dropped from Bedrock IDs starting with Sonnet 4.6.*
14. **Claude API IDs** — for example `claude-opus-4-6`, not Bedrock-style — and it follows **Anthropic's** first-party deprecation schedule, not Bedrock's. *It is on AWS and it is not Bedrock; that is the whole question.*
15. **Active** (fully supported, recommended), **legacy** (no further updates, may be deprecated), **deprecated** (functional but not recommended, has a replacement and a retirement date, and is likely less reliable), **retired** (requests fail). *"Deprecated still works" is true and insufficient — reliability is explicitly not guaranteed.*
16. **At least sixty days** before retirement, for publicly released models, by email and in the documentation. *Notice is the commitment; reading it is your problem.*
17. **Claude Console → Usage → Export**, then read the CSV, which breaks usage down by API key and by model. *This is the documented procedure and it is the only reliable inventory.*
18. That the model **will not retire before** that date. It is a floor, not a support commitment, and not a promise that it stays available afterwards. *"Guaranteed available until" is the wrong reading.*
19. `max_input_tokens`**,** `max_tokens`**, and a** `capabilities` **object** for every available model. *Use it to reconcile a checked-in registry rather than to replace one.*
20. Models within **Project Glasswing**, offered for **defensive cybersecurity workflows**. Mythos 5 shares Fable 5's specs and pricing. Access is **invitation-only with no self-serve sign-up**, via an Anthropic, AWS or Google Cloud account team. Mythos Preview is deprecated in favour of Mythos 5. *So: no, not without an invitation.*

14.32 Ten Practical Exercises

1. Extend the registry with a `capabilities` field and write the reconciliation job against the Models API described in section 14.21.

2. Add a `deprecation_report()` that prints every model in your codebase with days remaining, sorted ascending, and exits non-zero inside thirty days.
3. Add Claude Opus 4.6, Sonnet 4.6 and Opus 4.5 pricing and context figures from the documentation, and note how many fields you had to look up rather than infer.
4. Write a `plan_migration(from, to)` that turns `diff()` output into a checklist of code changes, evaluation runs and cost re-projections.
5. Add platform awareness: a `platform` argument that selects the right ID format and the right retirement calendar.
6. Build the twelve-month cost projection from Lab 5 as a function over the registry, with the Sonnet 5 step included.
7. Add a check that flags any request whose estimated input exceeds the model's context, using Chapter 6's `count_tokens` endpoint for the estimate.
8. Write a property test asserting that no model's `max_output` exceeds its `context`, and that every deprecated model has a replacement.
9. Add `legacy` as a fourth status and decide what preflight should do with it. Justify your choice.
10. Make `preflight()` emit structured JSON suitable for a CI annotation, with file and line where the request shape was built.

14.33 Five Challenge Labs

1. **The reconciliation service.** Run the Models API diff on a schedule, store the history, and produce a timeline of when each fact about each model changed. Answer: how often does a fact you depend on actually move?
2. **Behavioural drift detection.** Pin a fifty-example evaluation set to one model ID and run it weekly for a quarter, with Wilson intervals. Report whether you detected anything, including if the honest answer is "no change was established" — Chapter 12's discipline applies.
3. **The multi-platform registry.** Extend the model to cover the Claude API, Bedrock, Claude Platform on AWS, Google Cloud and Microsoft Foundry, with per-platform IDs, endpoint types and retirement calendars. Then find a model whose status differs between two of them.
4. **Automated migration.** Given `diff()` output, generate a pull request that mechanically applies the safe changes (removing `temperature`, rewriting the thinking block) and flags the ones needing judgement. Measure what fraction is mechanical.
5. **The cost-and-capability frontier.** For one of your own workloads, measure quality with intervals on all four current models, price each with `cost()`, and plot the frontier. Identify which models are dominated. This is the empirical half of Chapter 15, run early.

14.34 Ten Interview Questions

1. What does pinning a model ID guarantee, and what does it not?
2. Why is `claude-sonnet-4-5` a different kind of string from `claude-sonnet-5`?
3. A pinned model starts producing different output. Walk me through your diagnosis.

4. Why do the reliable knowledge cutoff and the training data cutoff differ, and when does the difference matter?
5. Your team wants to move from Sonnet 4.5 to Sonnet 5. What is on the checklist?
6. How would you find out which Claude models your organisation is calling right now?
7. Why is "we run on Bedrock" relevant to a deprecation conversation?
8. Design the registry. What fields, and what happens when a field is unknown?
9. A model you depend on is retired and you need to explain a decision it made last year. What do you have?
10. Why is model selection and effort selection one decision rather than two?

14.35 Summary

Part 3 opens with a catalogue and a clock. Four models are generally available on the Claude API: Fable 5 as the most capable widely released model at $10/$50 per million tokens, Opus 5 as the documentation's recommended starting point for complex agentic and enterprise work at $5/$25, Sonnet 5 at $2/$10 introductory pricing, and Haiku 4.5 at $1/$5. The first three carry a one-million-token context and a 128,000-token output ceiling; Haiku 4.5 carries 200,000 and 64,000, is a generation behind, and is the only current model whose reliable knowledge cutoff — February 2025 — differs from its training data cutoff. Output is priced at exactly five times input across the entire family, which is what makes Chapter 13's thinking arithmetic bite: an eight-thousand-token trace turns a $35,000 monthly Opus 5 workload into $235,000, a larger swing than the whole ten-fold spread from Haiku to Fable. Two further models, Mythos 5 and Mythos Preview, exist inside Project Glasswing for defensive cybersecurity work and are invitation-only, which is a useful reminder that appearing in a table is not the same as being procurable.

The most consequential knowledge in the chapter is about strings. Every Claude model ID is a pinned version, and from the 4.6 generation onward those IDs are dateless — `claude-opus-5`, `claude-sonnet-4-6` — which the documentation explicitly flags as a source of misunderstanding, because a dateless ID looks like an evergreen pointer and is not one. It is the snapshot. Only pre-4.6 short forms such as `claude-sonnet-4-5` are convenience pointers, resolving to a dated snapshot. Believe the wrong thing in either direction and you will either wait for an improvement that never arrives or pin something that can move. Pinning is also a bounded guarantee: weights are fixed for an ID, but the serving infrastructure around it — router, safety classifiers, sampling logic — can change, and the documentation names an infrastructure update as the likeliest cause of unexplained drift on a stable ID. The only instrument that will notice is a regression suite pinned to that ID, which is Chapter 8's evaluation work becoming Chapter 24's standing practice.

Capability does not travel in a bundle, and this is where the avoidable errors live. Claude Sonnet 5 supports adaptive thinking and all five effort levels and does not support task budgets. Claude Haiku 4.5 is the only current model using extended thinking, supports no `effort` parameter at all, and has no interleaved thinking. Neither Fable 5 nor Haiku 4.5 supports the 300K batch output beta that six other models do. `temperature`, `top_p` and `top_k` are deprecated from Opus 4.7 onward and return a 400 on non-default values while remaining in the SDK request types, so the failure is at runtime and never at compile time. Read the matrix, not the tier — and encode it, because a matrix in prose is a matrix nobody checks. The same model is addressed differently on five platforms, and two of them, Amazon Bedrock

and Google Cloud, run their own retirement calendars; Claude Platform on AWS runs on AWS, uses Claude API IDs, and follows Anthropic's.

The clock is the part teams underestimate. Five models retired in the six months to July 2026. Anthropic commits to at least sixty days' notice before retiring a publicly released model, and the introduction's team received exactly that notice at an email address belonging to someone who had left in March, leaving them seven days and a migration they had not costed. The lifecycle vocabulary is precise — active, legacy, deprecated, retired — and deprecated carries an explicit reliability caveat rather than merely a countdown. Active models carry "not sooner than" floors, which are the earliest the door can close and not a commitment that it stays open; two current models sat inside three months of theirs at the time of writing. The documented inventory tool is the Console usage export, a CSV broken down by API key and model, and it is the only way to find the batch job nobody remembers deploying.

Which leaves the mini project, whose whole content is one move: turn the string into a record. Sixteen models with provenance on every field, where a value is either verified on a stated date or `UNKNOWN` and `UNKNOWN` raises rather than defaulting; twelve preflight checks, each reproducing a documented 400 or an operational hazard; a migration differ that reports fields it cannot compare instead of quietly omitting them; and forty-one tests that fail when the documentation moves. Run against the introduction's one-line model bump, it returns eleven differences, two of which break every request and one of which changes the bill on the first call. Run against nine requests a real team would send, eight fail and it says why, in the region of a millisecond, offline. Chapter 15 turns the catalogue into a selection procedure with measurements on one side and Chapter 11's costs on the other; this chapter's contribution is that when that procedure produces an answer, changing to it is one line and a test run rather than an archaeology exercise.

KEY TAKEAWAYS

1. **Four current models**: Fable 5, Opus 5, Sonnet 5 (all 1M/128K) and Haiku 4.5 (200K/64K, a generation behind).
2. **Output costs exactly 5× input** across the family — $1/$5 to $10/$50, a ten-fold spread end to end.
3. **A dateless 4.6+ ID is a pinned snapshot**, not a pointer to the latest. Only pre-4.6 short forms are aliases.
4. **Pinning fixes weights, not behaviour.** Serving infrastructure moves; a regression suite is the only detector.
5. **Capability does not travel in a bundle.** Sonnet 5 has no task budgets; Haiku 4.5 has no `effort`.
6. **Two cutoffs, not one**, and they differ on Haiku 4.5: reliable Feb 2025, training data Jul 2025.
7. **Bedrock and Google Cloud set their own retirement calendars.** Claude Platform on AWS follows Anthropic's.
8. **Sixty days' notice is the commitment**; "not sooner than" is a floor, not a guarantee of availability.
9. **Sonnet 5's price rises 50% on 1 September 2026** — a scheduled change larger than most optimisations.
10. **Turn the string into a record with provenance.** Unknown must raise, not default; 41 tests, no network.

14.36 Glossary

Term	Definition
Active	A model that is fully supported and recommended for use.
Alias	A pre-4.6 convenience pointer resolving to the most recent dated snapshot for a minor version.
Batch output beta	`output-300k-2026-03-24`; raises the Batches API output ceiling to 300K on six models.
Dateless ID	The 4.6-generation-and-later format, for example `claude-opus-5`. A pinned snapshot, not a pointer.
Deprecated	Functional but not recommended; has a replacement and a retirement date, and is likely less reliable.
Legacy	No longer receiving updates; may be deprecated in future.
Models API	`GET /v1/models`; returns `max_input_tokens`, `max_tokens` and a `capabilities` object per model.
Pinned snapshot	A model ID whose underlying weights and configuration are fixed for the lifetime of the ID.
Project Glasswing	The invitation-only programme covering Claude Mythos 5 and Mythos Preview, for defensive cybersecurity.
Provenance	A record of whether a registry field was verified, and when. The alternative to guessing.
Reliable knowledge cutoff	The date through which a model's knowledge is most extensive and reliable.
Retired	No longer available; requests fail.
Retirement floor	A "not sooner than" date: the earliest a model can retire, not a commitment to availability.
Serving infrastructure	Router, safety classifiers and sampling logic around fixed weights; can change under a pinned ID.
Tier	A family position — Fable, Opus, Sonnet, Haiku — indicating a trade-off, not nested capability.
Training data cutoff	The broader date range of a model's training data, distinct from the reliable knowledge cutoff.

14.37 Further Reading

Primary sources. Five pages carry everything in this chapter and all five change: `platform.claude.com/docs/en/about-claude/models/overview` for the family table, `.../models/model-ids-and-versions` for the pinning rules and the dateless misconception, `.../model-deprecations` for the lifecycle, the status table and the parameter deprecations, `.../models/migration-guide` for the version-to-version upgrade paths, and `.../about-claude/pricing` for the complete rate card including batch and caching. The Models API reference at `.../api/models/list` is the programmatic form. Anthropic's *Commitments on Model Deprecation and*

Preservation sets out the weight-preservation position discussed in section 14.11, and the Transparency Hub is the source for how the two cutoff dates are defined. Read the release notes for each model — *What's new in Claude Opus 5*, *What's new in Claude Sonnet 5*, *Introducing Claude Fable 5 and Claude Mythos 5* — before any migration; they contain the API changes that a specification table does not.

Where to go next in this book. Chapter 15 is the direct sequel: this chapter is the catalogue, that one is the selection procedure, with Chapter 8's intervals on one side and Chapter 11's costs on the other. Chapter 16 is the Messages API in full, where the model ID becomes a request. Chapter 17 covers streaming, vision, files and documents — the input modalities every model in the table supports. Chapter 18 is extended and adaptive thinking as an API surface, expanding section 14.8's matrix into working code. Chapter 19 covers rate limits, pricing and quotas, which is section 14.7 as an operational constraint rather than a table. Chapter 24 builds the regression suites that detect the serving-infrastructure drift of section 14.4, and Chapter 29 covers the errors and retries that a 400 from section 14.18 becomes in production. Chapter 57 is the observability that makes a retired model's decisions defensible, and Chapter 61 covers the compliance, governance and data-residency obligations that make platform and endpoint choice a contractual question rather than a technical one.